Second Sig: UI spec

Oct 22, 2026 · @Rafi

One local page that makes the argument to a non-crypto judge in two clicks per scenario: Judge, then Execute.

Purpose and stack

The page exists for one three-minute demo in front of judges who are mostly AI engineers. Every element either shows what the bytes do, shows what the judge decided, or explains a crypto concept the moment it's needed. Anything else is cut.

Stack. Vite + React + TypeScript + Tailwind for the page. FastAPI with `sse-starlette` for the judge, which also serves the built page, so the demo is two processes: `anvil` and `uvicorn`. One `useJudgeStream` hook turns the server's events into card state; there is no client-side state beyond that and the selected scenario.

Non-goals. No wallet connect: the owners' test keys live on the server. No auth, no persistence, no mobile layout, no dark/light toggle. One 1440 px laptop screen mirrored to a projector, tested at 125% browser zoom so the back row can read it.

Design rules

A high-trust financial interface: monochrome, flat, quiet, with colour reserved for the verdict. The references are Linear, Stripe's dashboard and Fireblocks, not a hackathon landing page.

Token	Value	Use
`bg`	`#0A0A0A`	Page
`surface`	`#141414`	Cards, top bar, banner
`border`	`#262626` (`#333333` on hover and focus)	Every edge, 1 px, never thicker
`text`	`#EDEDED`	Primary text, the one primary button (white on black)
`text-2`	`#A3A3A3`	Labels, secondary text, reasoning stream
`text-3`	`#6B6B6B`	Captions, timestamps, placeholders
`match`	`#3DBF7A` on `#0F2A1C`	MATCH badge, executed state, info dots

Token	Value	Use
`ask`	`#D9A441` on `#2A2008`	ASK badge, notable rows
`reject`	`#D14D4D` on `#2A1010`	REJECT badge, critical rows, revert line

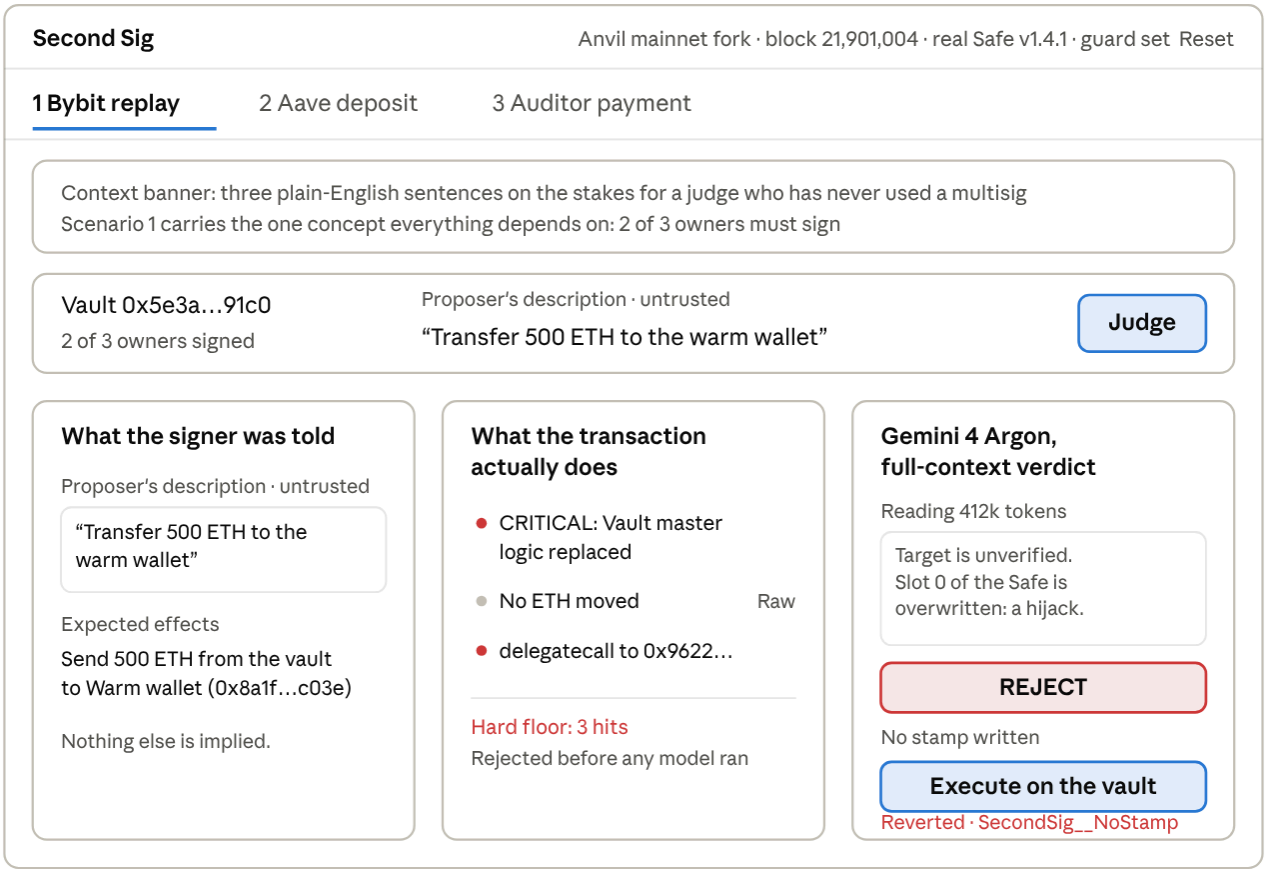
Type. Inter (or Geist Sans) at 14 px body and 16 px in cards; headings 20 and 24 px semibold, sentence case, no letter-spaced uppercase except 11 px labels. JetBrains Mono (or Geist Mono) at 13 px for addresses, hashes, calldata, slots and amounts. Line height 1.5.

Rules.

- No gradients, no glows, no drop shadows, no emojis, no decorative icons. The only icon is the 6 px status dot inside a badge or row.
- One colour per state and never for decoration: emerald means MATCH or executed, amber means ASK or notable, crimson means REJECT, critical or reverted. Everything else is grey.
- Addresses show as `0x9622…7242` in mono, full value on hover, copy on click. Known addresses show their address-book name first: `Warm wallet (0x8a1f…c03e)`.
- Amounts show with thousands separators and the symbol: `10,000 USDC`, `500 ETH`, never raw wei.
- Plain English first, hex second. Raw trace entries are available behind a "Raw" toggle, collapsed by default.
- One primary button per card at most. Secondary actions are text buttons.
- Nothing animates except the reasoning stream and a 150 ms opacity fade on state changes. No spinners; loading states are skeleton rows in `border` grey.
- 8 px spacing grid; cards have 24 px padding and 8 px radius; the three cards are equal width with 16 px gutters.

Page layout

The page: one banner, one proposal, three cards



page wireframe · Bybit scenario after Judge and Execute · drawn in the doc's own greys, the real page uses the tokens above

One screen, no scrolling at 1440 px: the top bar states what is real, the banner teaches the one concept, the proposal strip shows what the humans signed, and the three cards fill left to right as the pipeline runs.

The three cards

The cards fill left to right as the pipeline runs, so the audience watches the argument assemble: what the signer was told, what the bytes do, what the judge concluded.

Card 1: What the signer was told. The description, quoted in a bordered block with an 11 px label `Proposer's description · untrusted`. Below it, `Expected effects`: one row per effect from the forward pass, in plain English with address-book names (`Send 500 ETH from the vault to Warm wallet`). Footer: `Nothing else is implied by this description.` when `nothing_else` is set. States: empty (`Run the judge to begin`), skeleton rows while the forward pass runs, filled.

Card 2: What the transaction actually does. One row per sentence of the backward narration, each with a severity dot and a `Raw` toggle that reveals the trace entry in mono. Critical rows take the `reject` tint and start with `CRITICAL:`. Below the rows, a `Hard floor` line: `No hard-rule hits` in `text-3`, or each hit in crimson with its one-line reason and the caption `Rejected before any model ran`. States: empty, skeleton, filled.

Card 3: Gemini 4 Argon, full-context verdict. Top to bottom:

1. Stage line in `text-3`: `Fetching source for 7 contracts` → `Reading 412k tokens of Solidity` → `Reasoning` → done. The token count is real, from `count_tokens`.
2. Reasoning stream: a 220 px scrollable area, 14 px `text-2`, auto-scrolling while streaming, with thought summaries (if available) in italics above the final reasoning.
3. Verdict badge, 24 px, full width: `MATCH`, `ASK` or `REJECT`, with one line under it: for ASK the question, for REJECT the first discrepancy, for MATCH `Every effect is explained by the description and the contract source.`
4. Stamp line, mono: `Stamp signed by judge 0x3f…9a12 · fork block 21,901,004 · attested in 0xab…` or `No stamp written`.
5. The primary button `Execute on the vault`, enabled in every verdict state, because the point is that execution is attempted and the chain decides.
6. Result line: `Executed · 0x4c… · Vault now holds 10,000 aUSDC` in emerald, or `Reverted by SecondSigGuard · SecondSig__NoStamp(0x7e…) · 2 of 3 owners had signed` in crimson.

Page state	Cards	Buttons
Idle	Banner and proposal shown, cards empty	`Judge` enabled
Judging	Cards 1 and 2 fill in parallel, card 3 shows the stage line	`Judge` disabled
Verdict	Badge, stamp line	`Execute on the vault` enabled
Executed or blocked	Result line	`Reset` returns the fork to the base snapshot
Model unavailable	Card 3 shows `Gemini unavailable` and a `Replay from log` text button	Replay re-emits the logged events; the stage line says `Replayed`

Plain-English translations

The known slots are labelled by code (`judge/labels.py`); the backward pass writes the sentence; the UI only colours by severity. Judges never need to know what a storage slot is to follow card 2.

What the trace shows	What card 2 says	Severity
Write to the Safe's slot 0	`CRITICAL: Vault master logic replaced with code at 0xbDd0…9516. Every future action runs the attacker's code.`	critical
Write to the Safe's guard or fallback-handler slot	`CRITICAL: Vault guard changed`	critical
Write to owners, threshold or modules	`Vault owners, signature threshold or modules changed`	critical
`delegatecall` to a non-allowlisted address	`CRITICAL: Vault ran someone else's code as if it were its own (delegatecall to 0x9622…7242, unverified)`	critical
`delegatecall` to MultiSendCallOnly	`Batched 2 actions (standard Safe batching)`	info
ETH balance change	`Vault sent 500 ETH to Warm wallet` or `No ETH moved`	notable
USDC `balances[]` change	`USDC moved: Vault −10,000, Aave Pool +10,000`	notable
USDC `allowed[vault][x]` set to max	`Spending permission: Auditor may now spend unlimited USDC from the vault, with no further signatures`	notable
Allowance set then consumed in the same transaction	`Temporary permission for Aave Pool, fully used`	info
aUSDC balance change	`Vault received 10,000 aUSDC (Aave's deposit receipt)`	notable
Aave reserve, index or rate writes	`Aave internal bookkeeping (interest index, rates)`	info

What the trace shows	What card 2 says	Severity
Unknown slot in a verified contract	`Unlabelled storage change in AavePool, slot 0x3a… (the judge reads the source)`	notable
Unknown slot in an unverified contract	`Storage change in unverified contract 0x…`	critical
Event emitted	`Event: Supply(USDC, Vault, 10,000)`	info

Context banners

Three sentences each, shown above the proposal, written for someone who has never used a multisig. The first scenario's banner carries the one concept everything else depends on.

1. Bybit replay. This vault is a Safe multisig: 2 of its 3 owners must sign before anything moves, and that is how most crypto treasuries are run. In February 2025, Bybit's signers approved what looked like a routine transfer; the bytes they signed instead rewrote the vault's master code and handed about $1.4 billion to the attackers. Below is the same attack, replayed against a real Safe on a fork of Ethereum, with Second Sig installed.

2. Aave deposit. Honest treasury work is messy: depositing USDC into Aave touches several contracts and changes dozens of storage values. A rule engine that panics at complexity would block legitimate work, and signers would learn to ignore it. Second Sig reads the actual Aave contract code and checks that every change is what a normal deposit does.

3. Auditor payment. This description is true but incomplete: the transaction pays the auditor 100 USDC and also grants them permission to spend every USDC the vault will ever hold, with no further signatures. No single rule catches this, because unlimited approvals are sometimes intended. Second Sig notices the gap between the words and the bytes, and asks.

The top bar's standing line. `Anvil mainnet fork · block 21,901,004 · real Safe v1.4.1 · guard set` so a judge can see at a glance that nothing is simulated in a spreadsheet.

API and event stream

Eight endpoints, one of them a server-sent event stream. The server owns the owner keys, the judge key and the fork; the page owns nothing.

Method and path	Does	Returns
`GET /api/state`	Fork and Safe facts for the top bar	fork block, Safe address, owners, threshold, guard, judge address, ETH and USDC balances
`GET /api/scenarios`	The three scenarios	id, title, banner copy, description
`POST /api/proposals/{scenario}`	Builds the Safe transaction from the fixture and signs it with two owners	`proposal_id`, `safe_tx`, `safe_tx_hash`, `description`, signer addresses
`POST /api/judge/{proposal_id}`	Starts the pipeline as a background job	`job_id`
`GET /api/judge/{job_id}/events`	SSE stream of the pipeline	events below
`POST /api/execute/{proposal_id}`	Runs the real `execTransaction` with the signatures	`executed` with tx hash and balances after, or `reverted` with decoded error name and args
`POST /api/reset`	`evm_revert` to the base snapshot, then re-snapshot	new fork block
`POST /api/judge/{proposal_id}/replay`	Re-emits the logged events for this proposal with no model call	`job_id`

Events on the stream. `stage {name}` for `rehearsing`, `floor`, `forward`, `backward`, `judging`, `stamping`, `done`; `floor {hits}`; `forward {ExpectedEffects}`; `backward {Narration}`; `thought {delta}`; `reasoning {delta}`; `verdict {Verdict}`; `stamp {stamp, attest_tx}`; `error {message, replay_available}`. Forward and backward run concurrently, so their events arrive in either order and each fills its own card.

How the reasoning streams. The Argon call runs with streaming on. `reasoning` is the first property in the `Verdict` schema, so the server watches the raw JSON text, and once `"reasoning": "` has arrived it forwards every following character, JSON-unescaped, as

`reasoning` deltas until the closing quote. The full JSON is parsed when the stream ends and emitted as `verdict`. If thought summaries are available on Argon, they arrive first as `thought` deltas. This works whatever the workshop says about the flags.

Decoding a revert. `execute` catches the revert data, matches the 4-byte selector against the guard's ABI (`SecondSig__NoStamp(bytes32)`, `SecondSig__StaleStamp(bytes32,uint64)`) and the Safe's own error strings (`GS013` and friends), and returns the name and args so the result line never shows raw hex.

Components and build order

Fourteen components, no component library, one hook. Everything the server sends is already in display form (names, formatted amounts, sentences), so components render and never compute.

Component	Holds
`App`	Selected scenario, `proposal`, `jobId`; renders the rest
`TopBar`	Wordmark, fork line from `/api/state`, `Reset`
`ScenarioTabs`	The three scenarios; switching calls `/api/reset` then `/api/proposals`
`ContextBanner`	Banner copy
`ProposalStrip`	`Address` for the Safe, signer dots (2 of 3 filled), `DescriptionQuote`, the `Judge` button
`ToldCard`	Expected effects rows, footer
`DoesCard`	`EffectRow` list with severity dot and `Raw` toggle, `FloorNotice`
`VerdictCard`	`StageLine`, `ReasoningStream`, `VerdictBadge`, `StampLine`, `ExecuteButton`, `ResultLine`
`Address`, `Badge`, `Button`, `Skeleton`	Primitives
`useJudgeStream(jobId)`	Opens the SSE connection, reduces events into `{stage, floor, forward, backward, thought, reasoning, verdict, stamp, error}`

Build order.

1. Friday: static page rendering a hand-written fixture JSON for each card, with the tokens applied. If it looks right with fake data, the rest is plumbing.
2. Friday: `useJudgeStream` against a fake endpoint that replays a recorded event list with delays.
3. Saturday morning: cards 1 and 2 on real events for the Bybit scenario.
4. Saturday afternoon: card 3 with the real stream, stamp line, `Execute`, result line, for Aave and Bybit.
5. Sunday: banners, tabs with reset, the auditor scenario, zoom check on the projector, `Replay from log`.

Definition of done. Each scenario runs Judge then Execute at 125% zoom with no console errors in under 90 seconds end to end, three times in a row, with the network off after the RPC cache is warm.